Informe TP7 — NoSQL y nuevas formas de almacenamiento | Food Store

Integrantes: Avalos Pablo, Blangetti Sofia, Suarez Ismael. **Materia:** Bases de Datos II — UTN. **Repositorio:** github.com/ismaelsuarez/foodstore_nosql_movil, versión de entrega: etiqueta `u7-cierre`. **Estado:** informe del 2026-11-06; la revisión del equipo no consta; sin aprobación docente.

Evidencias en la etiqueta `u7-cierre`, bajo `unidades/unidad-7/`: **E1** `evidencias/ejecucion_20261106_mongo9.txt` · **E2** `docs/laboratorio_migracion.md` · **E3** `docs/inventario_origen_entorno.md` · **E4** `datos/pedidos_origen.json` · **E5** `evidencias/ejecucion_20261106.txt` · **E6** `docs/contraste_ia.md`. Las salidas completas se conservan fuera del repositorio; los extractos E1 y E5 indican su procedencia.

A. Modelo y alcance

El trabajo continúa el modelo relacional de `foodstore_baseII`, etiqueta `u6-cierre`, commit `8f7877c8a1ff25c1be28b29c46c38cd7c9663e86`. El modelo de origen tiene cinco tablas: `categoria`, `usuario`, `producto`, `pedido` y `detalle_pedido`. El subconjunto migrado es «pedidos y detalle de pedidos»: los pedidos 1–5 y sus 7 líneas, leídos de la base `foodstore` (PostgreSQL 17.11) con datos del usuario y del producto. El destino es la colección `foodstore_u7.pedidos`, con un documento por pedido. Las decisiones de modelo son del equipo, no de la consigna (E3).

PostgreSQL	MongoDB	Tipo BSON	Regla
`pedido.id`	`_id`	Long	ID del origen, construido desde texto
`pedido.fecha` (DATE)	`fecha`	string	Fecha comercial `YYYY-MM-DD`; no se usa `created_at`
`estado`, `forma_pago`	iguales	string	Valor exacto del ENUM (`TERMINADO`, `EFECTIVO`…)
`pedido.total`	`total`	Decimal128	Copiado del origen y verificado (RI-02)
`usuario.id`, `nombre`, `apellido`	`usuario.{usuario_id, nombre, apellido}`	Long, string	Cliente embebido; solo datos esenciales
`detalle_pedido.id`, `producto_id`	`items[].detalle_id`, `producto_id`	Long	IDs conservados
`producto.nombre`	`items[].producto_nombre`	string	Instantánea descriptiva; los filtros usan `producto_id`
`cantidad` (INTEGER)	`items[].cantidad`	Int32	Entero explícito
`precio_unitario`, `subtotal`	`items[].precio_unitario`, `subtotal`	Decimal128	Precio **histórico** de la línea, no `producto.precio`
`eliminado` (pedido y línea)	`eliminado`, `items[].eliminado`	bool	Bajas lógicas conservadas

Por qué se embebe. Las cuatro consultas filtran y devuelven pedidos como unidad. C4a lista los pedidos del lote con una proyección parcial (`_id` y `estado`); C4b filtra por estado; C4c busca un producto dentro de las líneas; C4d filtra por total con otra proyección parcial (`_id` y `total`). Ninguna de las dos devuelve el pedido completo. El embebido responde al acceso frecuente declarado, que es recuperar el pedido completo: con el cliente y las líneas en el mismo documento, ese pedido se obtiene en una sola lectura y se escribe de forma atómica *por documento*. El costo es duplicar nombres de cliente y de producto como copia, sin actualización automática.

Bajas lógicas. Se conservan todos los pedidos y todas las líneas con su indicador. C4b, C4c y C4d exigen `eliminado: false` en el pedido, y C4c exige además una línea vigente del producto en el mismo elemento. El total es la suma de los subtotales de las líneas no eliminadas. En este lote no hay bajas lógicas (E3, E4).

Datos. Los usuarios 1–3 son sintéticos del seed U6: Ana Gómez, Luis Paz y Marta Ruiz. Se compararon con el seed antes de exportarlos. No se trasladan mail, celular, contraseña ni rol (E4).

Representación del modelo, armada con los valores del pedido 1 de E4. No es una salida de `mongosh`:

```
{ _id: Long("1"), fecha: "2026-03-01", estado: "TERMINADO", forma_pago: "EFECTIVO",
  total: Decimal128("2800.00"), eliminado: false,
  usuario: { usuario_id: Long("1"), nombre: "Ana", apellido: "Gómez" },
  items: [ { detalle_id: Long("1"), producto_id: Long("1"), producto_nombre: "Muzzarella",
             cantidad: Int32(2), precio_unitario: Decimal128("1000.00"),
             subtotal: Decimal128("2000.00"), eliminado: false },
           { detalle_id: Long("2"), producto_id: Long("3"), producto_nombre: "Coca 1.5L",
             cantidad: Int32(1), precio_unitario: Decimal128("800.00"),
             subtotal: Decimal128("800.00"), eliminado: false } ] }
```

B. Migración e integridad

Componente	Observado	Evidencia
Origen	PostgreSQL 17.11, base `foodstore`, contenedor `foodstore-u6-evaluacion` (red `none`); solo lectura	E3, E5
Destino	MongoDB 9.0.4, imagen `mongo@sha256:452bdaf2…a058`, `mongosh` 2.14.0; standalone, red `none`, sin puertos publicados	E1, E2
Acceso	`u7_operador` con `readWrite` solo sobre `foodstore_u7`; carga y consultas en sesiones como `u7_operador`. La misma lectura sin autenticación se rechazó: `requires authentication`, código 1	E1

1. `postgresql/extraccion_pedidos.sql`: transacción `REPEATABLE READ, READ ONLY` con `ROLLBACK`. Antes de exportar comprueba el lote completo, la ausencia de referencias faltantes, RI-01, RI-02 y que los usuarios sean los del seed sintético. Genera `datos/pedidos_origen.json` con IDs e importes como texto, sin pasar por números de punto flotante: 5 pedidos y 7 líneas, SHA-256 `930562ab…bec7a` (E2, E5). El origen se volvió a leer antes y después de la carga y coincidió con el archivo (E1).
2. `mongodb/migracion_pedidos.js`: valida el archivo y el lote, comprueba que ningún `_id` exista y carga con `insertMany` ordenado (`w: 1, j: true`). Resultado: `acknowledged=true insertados=5`, código 0 (E1).
3. `mongodb/validaciones_mongodb.js`: consulta los documentos almacenados. Las 12 comprobaciones pasaron (12 OK, 0 FALLO), con código 0 (E1).

Comprobación sobre lo almacenado	Resultado observado	Evidencia
Conteos e identificadores	5 pedidos, 7 líneas; pedidos 1–5, usuarios 1–3, detalles 1–7, productos 1–3	E1
Tipos con `$type`	`long`, `int`, `decimal`, `string` y `bool`: 0 documentos con tipos inesperados	E1
Correspondencia con E4	Los 5 documentos coinciden campo por campo (EJSON canónico, con escala decimal)	E1
RI-01: cantidad Int32 > 0 e importes ≥ 0 en todas las líneas, incluidas las eliminadas	0 documentos con líneas inválidas	E1
RI-02: subtotal = cantidad × precio histórico; total = suma de subtotales no eliminados (o 0)	0 documentos incoherentes	E1
Precios históricos del producto 1	Pedido 1: 1000.00; pedidos 3 y 5: 1050.00. El precio actual del catálogo es 1050.00 (E3)	E1, E3

Validación de la carga frente a restricciones permanentes. RI-01 y RI-02 se verificaron con scripts sobre esta carga. La colección no tiene validador `$jsonSchema` ni otras reglas en el servidor, salvo el índice único de `_id`. Por eso estas comprobaciones no impiden escrituras inválidas futuras. Decimal128 representa los importes con exactitud, pero no valida reglas de negocio.

Incidente de entorno. El primer intento usó `mongo:8.0` (MongoDB 8.0.34), que terminó con código 1 al arrancar. Su comprobación de kernel rechazó el kernel Ubuntu `7.0.0-42`, aunque corresponde al upstream 7.0.14 (SERVER-136848). El equipo eligió `mongo:9.0.4`, que incluye la corrección SERVER-131779 y arrancó sin errores. El contenedor fallido se conserva detenido y sin uso (E2, E5).

C. Consultas y comparación

Líneas 23, 31, 36–39, 45–48 y 53–56 de `mongodb/consultas_mongodb.js`, copiadas sin cambios:

```
const lote = { _id: { $in: ['1', '2', '3', '4', '5'].map((id) => NumberLong(id)) } };
const c4a = pedidos.find(lote, { _id: 1, estado: 1 }).sort({ _id: 1 }).toArray();
const c4b = pedidos.find(
  { ...lote, eliminado: false, estado: 'TERMINADO' },
  { _id: 1 }
).sort({ _id: 1 }).toArray();
const c4c = pedidos.find(
  { ...lote, eliminado: false, items: { $elemMatch: { producto_id: NumberLong('1'), eliminado: false } } },
  { _id: 1 }
).sort({ _id: 1 }).toArray();
const c4d = pedidos.find(
  { ...lote, eliminado: false, total: { $gt: NumberDecimal('3000.00') } },
  { _id: 1, total: 1 }
).sort({ _id: 1 }).toArray();
```

Fragmento de `postgresql/consultas_postgresql.sql` (C4c; C4b usa el mismo lote y el mismo filtro de baja, con `p.estado = 'TERMINADO'`):

```
SELECT 'C4c' AS consulta,
       count(*) AS cantidad,
       string_agg(p.id::text, ',' ORDER BY p.id) AS pedidos
FROM pedido p
WHERE p.id IN (1, 2, 3, 4, 5)
  AND p.eliminado = FALSE
  AND EXISTS (
      SELECT 1
      FROM detalle_pedido dp
      WHERE dp.pedido_id = p.id
        AND dp.producto_id = 1
        AND dp.eliminado = FALSE);
```

Las consultas PostgreSQL corren en una transacción `REPEATABLE READ, READ ONLY` con `ROLLBACK`. `EXISTS` cuenta cada pedido una sola vez; en MongoDB, `$elemMatch` exige el producto y la línea vigente en el mismo elemento.

Consulta en MongoDB	Cantidad	Pedidos observados
C4a: todos los del lote, con ID y estado	5	1, 2, 3, 4, 5 (todos `TERMINADO`)
C4b: estado `TERMINADO`, no eliminado	5	1, 2, 3, 4, 5
C4c: producto 1 en una línea vigente	3	1, 3, 5
C4d: `total > 3000.00`, no eliminado	2	3 (3150.00), 4 (6200.00)

Comparación	Cantidad en PostgreSQL	Cantidad en MongoDB	IDs	Coinciden los IDs
Estado seleccionado (C4b)	5	5	1, 2, 3, 4, 5	Sí
Producto seleccionado (C4c)	3	3	1, 3, 5	Sí

Ambos archivos de consultas terminaron con código 0 (E1).

Umbral de C4d. Los totales del lote son 1050.00, 1500.00, 2800.00, 3150.00 y 6200.00; el promedio es 2940.00 (E3). 3000.00 es un importe redondo apenas superior al promedio y ningún total es exactamente 3000.00, de modo que no hay ambigüedad entre > y ≥.

Carga repetida. Una segunda ejecución de `migracion_pedidos.js` con el mismo archivo se rechazó antes de escribir: `RECHAZADO: ya existen los pedidos 1,2,3,4,5; no se inserta nada.`, código 3. Las validaciones posteriores dieron código 0, la colección siguió con 5 documentos y la huella SHA-256 del contenido se mantuvo (`7a2351dc…`) (E1). No se usan upserts ni borrados. `insertMany` no es atómico para el lote, y el caso de fallo parcial no se probó.

D. Contraste de IA y conclusión

Consulta (consigna 6). El prompt pidió comparar PostgreSQL y MongoDB para los pedidos de Food Store. Describía el modelo relacional de cinco tablas con claves, restricciones y lógica para subtotales y totales, y la colección con cliente e items embebidos, IDs Long, Int32, Decimal128 y precios históricos. Mencionaba el acceso frecuente al pedido completo y la necesidad de cantidades positivas, subtotales coherentes y total de líneas vigentes. Pedía ventajas, costos y responsabilidades de integridad, separar las capacidades del motor de lo implementado y no afirmar rendimiento no medido ni carencias de transacciones o validación. El texto completo y la respuesta están en E6.

La respuesta la generó Claude Opus 5.5 el 2026-11-06, en la misma sesión del asistente que implementó la migración. Por eso no es una consulta independiente. Se guardó antes de leer las fuentes del contraste.

Afirmación de la respuesta	Fuente y qué respalda	Evaluación
A1. `insertMany` con varios documentos no es atómico para el lote; la atomicidad entre documentos requiere transacciones en un replica set o un clúster fragmentado; una instancia standalone no las admite.	F1: cada documento es atómico, pero «the operation as a whole is not atomic». F6, sección *Availability*: «standalone deployments do not support transactions». F1 no dice literalmente que lo ya insertado permanece.	Respaldada documentalmente; no ensayada: no se probó un fallo parcial ni una transacción. El primer veredicto fue «necesita matiz», sin F6.
A2. Un `CHECK` evalúa solo la fila insertada o actualizada; no puede comparar `pedido.total` con la suma de `detalle_pedido`, que depende de lógica programable.	F5: PostgreSQL no admite `CHECK` que referencien datos distintos de la fila comprobada. Los `CHECK` de `schema.sql` cubren reglas de una fila (`cantidad > 0`, importes ≥ 0). El total agregado lo mantienen las funciones y los disparadores del TPI (`tpi/sql/objetos_programables.sql`), conservados en `u6-cierre`, que declaran límites ante DML concurrente directo y escritura directa del total.	Respaldada. Consecuencia: el diseño copia el total y lo verifica (RI-02), sin recalcularlo.

Fuentes oficiales, consultadas el 2026-11-06: F1 MongoDB Manual, *Atomicity and Transactions*, sección *Details*: mongodb.com/docs/manual/core/write-operations-atomicity/ · F5 PostgreSQL 17, *5.5. Constraints*, nota de *Check Constraints*: postgresql.org/docs/17/ddl-constraints.html · F6 MongoDB Manual, *Production Considerations*, sección *Availability*: mongodb.com/docs/manual/core/transactions-production-consideration/. También se usaron F2 *Transactions*, F3 *Schema Validation* y F4 *Embedded Data* (E6).

Conclusión (consigna 7) — propuesta pendiente de revisión del equipo; 186 palabras.

Proponemos mantener PostgreSQL como fuente principal de los pedidos y usar MongoDB solo como copia documental de lectura de este subconjunto, si el acceso al pedido completo lo justifica.

Integridad: en PostgreSQL, las claves foráneas y las restricciones CHECK rechazan en cada escritura líneas sin producto o cantidades no positivas, y los disparadores del TPI mantienen el total agregado, con límites declarados. En la colección, RI-01 y RI-02 se verificaron con scripts después de la carga; no se instalaron validadores permanentes.

Forma de acceso: cada documento contiene cliente e items, devuelve el pedido completo en una sola lectura y se escribe de forma atómica por documento, no por lote. Se conservaron IDs Long, cantidades Int32, importes Decimal128 y precios históricos; C4b y C4c devolvieron los mismos pedidos en ambos motores.

Duplicación: los nombres de cliente y de producto son copias; la coincidencia entre motores se verificó el 2026-11-06 y no implica sincronización.

El lote tiene cinco pedidos, un solo estado y ninguna baja lógica, y no se midió rendimiento. Ampliar el uso de MongoDB exigiría pruebas con más volumen y una carga reproducible desde el origen.

Uso de IA y decisiones humanas. Kiro se usó para especificar y Claude para implementar, documentar, generar la respuesta contrastada y redactar este informe; ambas herramientas fueron autorizadas por el docente. El equipo decidió, mediante encargos explícitos, el modelo, el lote y sus parámetros, el uso de MongoDB 9.0.4 y cada autorización de ejecución. El contraste, la conclusión y este informe siguen pendientes de revisión del equipo.

Limitaciones.

- Cinco pedidos, un solo estado (`TERMINADO`) y ninguna baja lógica: C4b devuelve el lote completo y los filtros sobre `eliminado` no se ponen a prueba.
- Sin mediciones de rendimiento.
- Sin validadores permanentes.
- Sin prueba de fallo parcial de `insertMany` ni de transacciones.
- La coincidencia entre motores es una verificación puntual, no una sincronización.
- La respuesta de IA no es independiente.

Referencias.

- **Consigna:** `BDII_Unidad7_TP_FoodStore_NoSQL.pdf`, «Trabajo Práctico · Unidad 7 — NoSQL y nuevas formas de almacenamiento | Food Store». Material de la cátedra; no está en el repositorio.
- **Entrega:** repositorio github.com/ismaelsuarez/foodstore_nosql_movil, etiqueta `u7-cierre`.
- **Unidad 7:** `README.md` (consignas, archivos y resultados) y `EVALUACION.md` (revisión y reproducción).
- **Consultas:** `mongodb/consultas_mongodb.js` y `postgresql/consultas_postgresql.sql`; extractos de ejecución E1 y E5.
- **Origen relacional:** github.com/ismaelsuarez/foodstore_baseII, etiqueta `u6-cierre`, commit `8f7877c8a1ff25c1be28b29c46c38cd7c9663e86`.